

# STL库-笔记

会：vector map

一般般：priority\_queue set

不会：bitset multiset unordered\_set unordered\_map

Vector内置函数：

```
push_back emplace_back pop_back  
back insert erase  
lower_bound upper_bound  
accumulate min_element max_element  
count find  
sort unique  
resize assign reserve  
reverse  
kth_element
```

//不太熟：

```
insert push_back push_front push emplace erase lower_bound upper_bound unique
```

没必要只能用 emplace\_back。常见可替代写法有好几种，效果等价

- 用 push\_back 加临时 pair：

```
g[u].push_back(pair<int,int>(v, w));
```

- 用 push\_back 和 make\_pair：

```
g[u].push_back(std::make_pair(v, w));
```

- 用 push\_back 与大括号（C++11 起）：

```
g[u].push_back({v, w});
```

- 仍然可以用 emplace\_back（原写法，会原地构造）：

```
g[u].emplace_back(v, w);
```

比较与建议：

- `emplace_back(v,w)` 是将参数原地转发给 `pair` 的构造函数，避免构造一个临时再拷贝/移动（在某些类型上能小幅提高性能）。但对于内含两个 `int` 的 `std::pair`，拷贝/移动开销极小，差异可以忽略。
- `push_back({v,w})` 简洁可读，推荐用在 C++11 及以上。
- 如果使用老编译器（C++98），用 `make_pair` 或显式 `pair` 构造。
- 小心：若容器元素类型有接受 `initializer_list` 的构造函数（不是 `pair` 的情况），`push_back({..})` 可能会产生歧义，但对 `pair<int,int>` 安全。

所以你可以任选其一。常见推荐是 `emplace_back` 或 `push_back({v,w})`。